

Linear Algebra in Reducing Multidimensional Data

To what extent is Principal Component Analysis effective in dimensionality reduction, and how does it compare mathematically and empirically to Linear Discriminant Analysis?

Subject: Math

Word Count: 3745

1 Introduction

Data analysis techniques are applied in math, statistical analysis, data science and data preprocessing. Applications of linear algebra optimize data by reducing the number of features in a dataset while retaining the most important information. Its applications range from machine learning models to quantitative finance, even breaking into quantum computing through quantum algorithms for parallel processing (Quantum PCA), in turn leading to higher accuracy and reduced complexity. This essay explores “To what extent is Principal Component Analysis (PCA) effective in dimensionality reduction, and how does it compare mathematically and empirically to Linear Discriminant Analysis (LDA)?” PCA will be put against LDA in reducing dimensionality of Iris and MNIST datasets, in order to test performance in low and high dimensional settings.

This essay was sparked by a deep passion for mathematical inquiry. I was drawn by a curiosity of the inner workings of high level algorithms in finance, data science and machine learning and the way in which they outperformed human capabilities by several orders of magnitude. In recent times, technology has become at the center of everyday life, through this I found it fascinating how everyday data can be interpreted through algorithms, giving profound insight. At the core, Algorithms are mathematical formulas, meaning modern innovation is no longer in optimizing physical processes, but computational ones. This essay explores the mathematical foundations of PCA and LDA, applies them to the Iris and MNIST datasets, and evaluates their performance through variance retention and class separation metrics.

2 Background and Mathematical Theory

2.1 Statistical Fundamentals

Let’s understand what is meant by dimensionality of a dataset. Consider the graph below where the points A, B, C, D can be positioned in a one dimensional number line from 0 to infinity:

	A	B	C	D
X	10	11	8	3

Now consider the graph when another test point is added, Y:

	A	B	C	D
X	10	11	8	3
Y	6	4	5	3

This graph is now one dimension greater with the introduction of Y, and can now be imagined as a Cartesian x-y axis grid with the points A, B, C, D. Likewise, we can add a third dimension Z and its data point, representing the data as a 3D X-Y-Z axis grid. With the introduction of a 4th, 5th, . . . dimension, the data can no longer be plotted visually. This is where we find PCA and LDA very useful, and what is meant by dimension reduction: the purpose is to reduce the columns while retaining the best projection of the data.

Three very important concepts in statistical analysis are the ideas of mean, variance, and covariance. Consider two sets of points: (4,5,6) and (1,5,9). The mean is simply an average of the numbers: sum over number of points. As we can observe, the two sets will have the same mean, although the variance is different. To compute variance, we consider the distance from each point to the mean. Accounting for negative numbers, the distance value is squared, and the sum is divided by the number of points. So in set one the variance is $\frac{2}{3}$ and in set 2 the variance is $\frac{16}{3}$. The higher variance value indicates the second set has a greater spread. This is easy to consider in one dimension, but how does this change with dimensions?

$$\text{Var}(X) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

As dimensions increase, we calculate the variance in each dimension: for 2 dimensions x-variance and y-variance, for 3 dimensions x-y-z, and so forth. This is very fundamental, but how do we distinguish

between sets of points showing the same variance? This is where covariance comes in. Covariance is essentially the sum of the product of coordinate deviations from their means. For example, consider two sets $\{(-2, 1), (0, 0), (2, -1)\}$ and $\{(-2, -1), (0, 0), (2, 1)\}$. Both sets have the same spread (variance), although their covariance differs as one is $\frac{4}{3}$ and the other is $-\frac{4}{3}$, indicating the trend of the points. A positive covariance suggests that as one variable increases, the other also tends to increase. This can be expressed as:

$$\text{Cov}(X, Y) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

$$\bar{x} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$

In matrix notation for an x-y dataset:

$$\begin{pmatrix} x_1 - \bar{x} & y_1 - \bar{y} \\ x_2 - \bar{x} & y_2 - \bar{y} \\ x_3 - \bar{x} & y_3 - \bar{y} \\ \dots & \dots \\ x_n - \bar{x} & y_n - \bar{y} \end{pmatrix}$$

2.2 PCA Mathematics

This ensures the first step in PCA, which is centering the set. This is done by taking the mean of each coordinate axis and displacing each point by that amount so that the mean of each axis is at the 0 point. The data is converted into a covariance matrix by multiplying the transpose of the centered data by the data. This is an important step in principal component analysis. Let C denote the covariance matrix:

$$C = \frac{1}{n-1} X^T X$$

Where an example transpose is shown below:

$$X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, X^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

n minus one is used because dividing by n can underestimate the covariance and create biased estimates. This technique is called Bessel's Correction. In the examples below are covariance matrices for x-y dimensions and n-dimensions. The matrix encompasses the covariance between each individual set of dimensions; in other words, how the variables vary independently and together:

$$\begin{pmatrix} \text{Var}(X) & \text{Cov}(X, Y) \\ \text{Cov}(Y, X) & \text{Var}(Y) \end{pmatrix}$$

For n elements it follows that the covariance matrix is:

$$\begin{pmatrix} \text{Var}(x_1) & \text{Cov}(x_1, x_2) & \text{Cov}(x_1, x_3) & \dots & \text{Cov}(x_1, x_n) \\ \text{Cov}(x_2, x_1) & \text{Var}(x_2) & \text{Cov}(x_2, x_3) & \dots & \text{Cov}(x_2, x_n) \\ \text{Cov}(x_3, x_1) & \text{Cov}(x_3, x_2) & \text{Var}(x_3) & \dots & \text{Cov}(x_3, x_n) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(x_n, x_1) & \text{Cov}(x_n, x_2) & \text{Cov}(x_n, x_3) & \dots & \text{Var}(x_n) \end{pmatrix}$$

It is important to understand the matrix first and foremost, and the relationship between the vector and the matrix. Consider a vector v as its components:

$$v = \langle v_1, v_2, v_3, \dots, v_m \rangle$$

We can view the vector as a matrix, changing it into column vector form:

$$\begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_m \end{pmatrix}$$

And interpret a matrix with n columns and m rows as n column vectors:

$$\begin{pmatrix} v_{1,1} & v_{2,1} & v_{3,1} & \cdots & v_{n,1} \\ v_{1,2} & v_{2,2} & v_{3,2} & \cdots & v_{n,2} \\ v_{1,3} & v_{2,3} & v_{3,3} & \cdots & v_{n,3} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ v_{1,m} & v_{2,m} & v_{3,m} & \cdots & v_{n,m} \end{pmatrix}$$

We can interpret matrices as linear transformations in the way that they can change a vector x . Consider this 2x2 matrix. R indicates the dimensionality of the matrix, meaning x must exist in 2 dimensions. x would not be computable as a 3x3 or 1x3 matrix, for example:

$$R^2 \rightarrow R^2 \quad T(x) = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} x$$

This transformation would be a 90-degree rotation of x since in any matrix multiplication we multiply the first row and the column to obtain a new number, which will be the first entry for the new vector in R^2 . The second entry in the new vector will be the product of the second row. For any given:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} u \\ w \end{pmatrix} = \begin{pmatrix} au + bw \\ cu + dw \end{pmatrix}$$

$$\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} u \\ w \end{pmatrix} = \begin{pmatrix} 0 \cdot u + (-1) \cdot w \\ 1 \cdot u + 0 \cdot w \end{pmatrix} = \begin{pmatrix} -w \\ u \end{pmatrix}$$

We look at the covariance matrix as a linear transformation, and in order to find our principal components we must find vectors x that will remain unchanged when the linear transformation is applied. These are called eigenvectors. Let $A = n \times n$ matrix and let X be a nonzero matrix $X \in R^n$ such that $AX = \lambda X$.

There is a scalar value of the matrix, called an eigenvalue, and X is the eigenvector. For example:

$$A = \begin{pmatrix} 0 & 5 & -10 \\ 0 & 22 & 16 \\ 0 & -9 & -2 \end{pmatrix} \begin{pmatrix} -5 \\ -4 \\ 3 \end{pmatrix} = \begin{pmatrix} -50 \\ -40 \\ 30 \end{pmatrix} = 10 \begin{pmatrix} -5 \\ -4 \\ 3 \end{pmatrix}$$

In this equation X is the column vector containing $(-5, -4, 3)$ and λ is 10. The concept of eigenvalue and eigenvector are fundamental in Principal Component Analysis since it is the eigenvectors that outline the most important (principal) components of data within the dataset, allowing for reduction from a multidimensional matrix to a number of eigenvectors, in turn reducing the size of the data while bringing forth the most important elements. Most matrices usually have more than one eigenvector, which indicates there are multiple principal components.

In addition, it is important to understand the concept of the determinant, denoted as $\det(X)$ in which X is a matrix describing some linear transformation. The determinant describes how the area of a unit square will change with a linear transformation. As described previously, we have seen how a matrix can describe a linear transformation: a stretch, rotation, shear, compression, or it can simply be reduced to a lesser dimension. The determinant describes how the area of a unit square changes and its orientation. So when:

$$x \in R^3, \quad \det(x) = 0$$

X is a matrix describing a transformation in which the matrix has been reduced to a 2D plane, a line, or a single point. Drawing from our definition of the eigenvalue we can derive the following equation, in which I is the identity matrix, which has ones down the diagonal, keeping vectors unchanged but allowing for linear algebra:

$$AX = \lambda X$$

$$(A - \lambda I)X = 0$$

Further, from previous knowledge of determinants we can use this to find intuitive solutions to this equation. As we know, the determinant represents a scaling value for linear transformations. This applies to matrix multiplication in which the composition of two matrices will have the same scaling factor as the product of both individual scaling factors. In other words:

$$\det(AB) = \det(A)\det(B)$$

Where A and B are both matrices. Going back to our equation:

$$AX = \lambda X$$

$$(A - \lambda I)X = 0$$

$$\det((A - \lambda I)X) = \det(A - \lambda I)\det(X) = 0$$

$$X \neq 0$$

$$\det(A - \lambda I) = 0$$

The determinant is computed using various methods. For an $n \times n$ matrix we can use the Laplace expansion defined by:

$$\det(X) = \sum_{j=1}^n (-1)^{i+j} x_{i,j} M_{i,j}$$

Where i is the row of the matrix and j is the column, $x_{i,j}$ is the entry in the i-th row and j-th column, and $M_{i,j}$ is the $(n-1) \times (n-1)$ submatrix of X obtained by removing the i-th row and j-th column. The indices j and i are interchangeable. Let's analyze the example for a 3 by 3 matrix:

$$\det \left(\begin{bmatrix} 1-\lambda & 2 & 3 \\ 4 & 5-\lambda & 6 \\ 7 & 8 & 9-\lambda \end{bmatrix} \right) = 0$$

$$\det \left(\begin{bmatrix} 1-\lambda & 2 & 3 \\ 4 & 5-\lambda & 6 \\ 7 & 8 & 9-\lambda \end{bmatrix} \right) = (1-\lambda) \cdot \det \left(\begin{bmatrix} 5-\lambda & 6 \\ 8 & 9-\lambda \end{bmatrix} \right) - 2 \cdot \det \left(\begin{bmatrix} 4 & 6 \\ 7 & 9-\lambda \end{bmatrix} \right) + 3 \cdot \det \left(\begin{bmatrix} 4 & 5-\lambda \\ 7 & 8 \end{bmatrix} \right)$$

$$\lambda = \frac{15}{2} \pm \frac{3\sqrt{33}}{2}, \quad 0$$

Now that we have identified the principal components of the dataset, we sort them by eigenvalue and project the data onto the principal components, reducing dimensions while keeping the most crucial data.

2.3 LDA Mathematics

On the other hand, Linear Discriminant Analysis deals with multiple classes or groupings in the data, with the overall goal of discriminating between classes. Let's first look at how LDA is performed on a two-variable dataset, then on an n-variable dataset. Consider the points: $X_1, X_2, X_3 \dots X_n \in R^d$ in which there are two classes C_1, C_2 . The approach is to find a unit vector $v \in R^d$ onto which if the data is projected will create the largest separation.

Let's consider the unit vector $v \in R^d$ which passes through the origin. Consider the projection of the points onto this vector in which a_i is the point on the vector and v^T is the vector transpose:

$$a_i = v^T X_i$$

Let $\mu_j = v^T m_j$ in which m_j is the j-th class's mean. Consider the following equation:

$$s_j^2 = \sum_{x_i \in C_j} (a_i - \mu_j)^2, \quad j = 1, 2$$

The equation represents the separations between the projected points and the mean of the class. This approach allows for consideration of variance, as only mean calculation can result in overlap and inaccuracy. This leads us to find a solution to the equation:

$$\max_{(v: \|v\|=1)} \frac{(\mu_1 - \mu_2)^2}{s_1^2 + s_2^2}$$

In which the numerator should be large and the denominator small. Next we must derive a formula for the distance between the two centroids (means). Firstly, it is important to understand the scalar quadratic form in which x is an n by 1 vector, A is an n by n matrix and x^T is the transpose of x (1 by n):

$$x^T A x$$

Consider this case where a is a 1 by n matrix:

$$(v^T a)^2 = \left(\sum_{i=1}^n v_i a_i \right)^2 = (v a^T)(v^T a) = v^T (a a^T) v$$

This solution mirrors the base case and occurs due to the multiplicity between v^T and a allowing for a to be a 1 by n matrix and a^T to be an n by 1 matrix. Let's apply this theory for derivation:

$$(\mu_1 - \mu_2)^2 = (v^T m_1 - v^T m_2)^2 = (v^T (m_1 - m_2))^2 = v^T (m_1 - m_2)(m_1 - m_2)^T v = v^T S_b v$$

Where:

$$S_b = (m_1 - m_2)(m_1 - m_2)^T \in R^{d \times d}$$

is the between-class scatter matrix. As described previously, this multiplication will result in a symmetric square n by n matrix. Let's further calculate the within-class scatter matrix:

$$s_j^2 = \sum_{x_i \in C_j} (a_i - \mu_j)^2 = \sum_{x_i \in C_j} v^T (x_i - m_j)(x_i - m_j)^T v = v^T \left[\sum_{x_i \in C_j} (x_i - m_j)(x_i - m_j)^T \right] v$$

Let:

$$S_j = \sum_{x_i \in C_j} (x_i - m_j)(x_i - m_j)^T$$

Lastly, consider:

$$S_w = S_1 + S_2 = \sum_{x_i \in C_1} (x_i - m_1)(x_i - m_1)^T + \sum_{x_i \in C_2} (x_i - m_2)(x_i - m_2)^T$$

This converts our initial problem into a more intuitive form, in which all S are n by n symmetrical matrices:

$$\max_{(v: \|v\|=1)} \frac{(\mu_1 - \mu_2)^2}{s_1^2 + s_2^2} = \frac{v^T S_b v}{v^T S_w v}$$

The maximizer in this equation will be the largest eigenvector of:

$$S_w^{-1} S_b v = \lambda v$$

For computational purposes, the equation is often written as:

$$S_b v = \lambda S_w v$$

Eigenvectors and eigenvalues are computed and the discriminatory direction becomes the eigenvector with the largest eigenvalue. Now let's standardize this process across n dimensions. Now let's consider multiclass Linear Discriminant Analysis. The same process is applied over c number of classes, with the same intuition applied: class means are spaced apart and class scatter is small, meaning the points are tighter together. The process in multiclass LDA is very similar to the base case of two classes.

In multiclass:

$$\sum_{j=1}^c s_j^2 = v^T \left(\sum_{j=1}^c S_j \right) v$$

$$S_w = \sum_{j=1}^c S_j$$

$$S_w = \sum_{j=1}^c \sum_{x_i \in C_j} (x_i - m_j)(x_i - m_j)^T$$

Likewise for the between-class scatter, although a detail does change as the mean of the projected centroids, defined as μ_j for class j, must take into account a global centroid of all classes. Since $c \geq 3$, each class might vary in size which a simple average can skew separation measures. Having more or less points, a weighted mean is introduced, ensuring that the between-class scatter factors in the distribution of the overall dataset. Let μ represent the weighted mean, n_j is the number of samples in class j, and n the total number of samples:

$$\mu = \frac{1}{n} \sum_{j=1}^c n_j \mu_j$$

The matrix changes as it is not the distance between individual classes that is maximized, but from the global centroid:

$$\sum_{j=1}^c n_j (\mu_j - \mu)^2 = v^T \left[\sum_{j=1}^c n_j (m_j - m)(m_j - m)^T \right] v = v^T S_b v$$

Similar process and result as with two-class LDA:

$$\max_{(v: \|v\|=1)} \frac{\sum_{j=1}^c n_j (\mu_j - \mu)^2}{\sum_{j=1}^c s_j^2} = \frac{v^T S_b v}{v^T S_w v}$$

The largest eigenvalues are found for the following problem and data is projected onto the corresponding eigenvectors:

$$S_w^{-1} S_b v = \lambda v$$

Computations would be as follows for an n by d data matrix X:

$$X \in R^{n \times d}$$

$$S_w = \sum_{j=1}^c \sum_{x \in C_j} (x - m_j)(x - m_j)^T$$

$$S_b = \sum_{j=1}^c n_j (m_j - m)(m_j - m)^T$$

$$S_b v = \lambda S_w v$$

Let V_k be all the nonzero eigenvectors ($k \leq c - 1$):

$$V_k = (v_1, \dots, v_k)$$

Data is projected as:

$$Y = X \cdot V_k \in R^{n \times k}$$

3 Methodology

In this analysis, the two methods were compared over an Iris dataset. The set was taken from the UC Irvine Machine Learning Repository. The dataset has 4 features (sepal length, sepal width, petal length, and petal width) and 150 samples, in addition to three classes (Iris Setosa, Iris Versicolour, and Iris Virginica), proving to be exceptional for the purposes of this essay, since we can analyze the differences between the two dimensionality reduction methods. Below is a small sample from within the dataset:

```
4.6,3.2,1.4,0.2,Iris-setosa
5.3,3.7,1.5,0.2,Iris-setosa
5.0,3.3,1.4,0.2,Iris-setosa
6.2,2.9,4.3,1.3,Iris-versicolor
5.1,2.5,3.0,1.1,Iris-versicolor
5.7,2.8,4.1,1.3,Iris-versicolor
6.3,3.3,6.0,2.5,Iris-virginica
5.8,2.7,5.1,1.9,Iris-virginica
7.1,3.0,5.9,2.1,Iris-virginica
6.3,2.9,5.6,1.8,Iris-virginica
```

The PCA dimensionality reduction was conducted on statistical analysis software R, on which the functions for PCA and LDA were built. Courtesy to the R-blogger website for posting open source code for both PCA and LDA, all sources are cited. This analysis will not cover the process in order to stay consistent with the math scope. But all code for is posted on GitHub.

A subsequent secondary test was done with the MNIST dataset. The MNIST Dataset consists of hand written 28 by 28 pixel grayscale images of handwritten number 0-9, each of the 784 pixels has a value between 0-255, indicating how dark the pixel is 0 (white) to 255 (black), the entire set has 60000 samples but for this paper, only 1000 were used in order to reduce computational downtime. The set was provided also via the UC Irvine ML Repository. This test was done for performance in high dimensional spaces, since now the set has 784 dimensions compared to the 4 in the Iris set

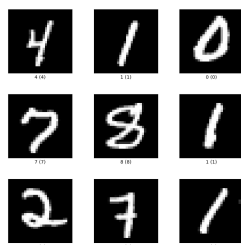


Figure 1: MNIST Data Sample

The Iris and MNIST datasets were run through the PCA and LDA functions, and then projected onto the first 2 principal components and discriminant components, in order to make a graphical visualization. Ellipses were added to the graph to visualize the data. This analysis will include empirical observations, as well as variance analysis for PCA only, since it is outside of LDA's scope.

The MNIST Dataset was first loaded from the UC Irvine ML repository, and then the data was preprocessed by flattening the 28×28 images into 784-dimensional vectors and standardizing them. Then the PCA and LDA functions were performed. For both PCA and LDA separation metrics: Between-Class distance, Within-Class scatter and separation ratio were calculated. For PCA exclusively the number of principal components needed to achieve 95% variance captured was also taken. Refer to bibliography for R code repository in GitHub

4 Analysis

4.1 Iris Dataset

The PCA analysis of the Iris dataset shows strong dimensionality reduction capabilities. The First principal component (PC1) captures 72.96% of the total variance, while the second component (PC2) accounts for 22.85%, yielding a cumulative variance of 95.81% with just two components. This Represents a 50% reduction in the dimensionality of the dataset, as a 4 dimensional set was reduced to 2 dimensions while retaining nearly all the information of the dataset, as seen with the 95% variance captured threshold

However, the plot shows visible class overlap, revealing a critical limitation. The PCA projection shows visible overlap between the versicolor (green) and virginica (blue) species, particularly in the region where PC1 ranges from 0 to 2. The setosa species (red) is well-separated, positioned distinctly in the negative PC1 region (centroid at $PC1 = -2.22$). This separation pattern occurs because PCA is an unsupervised technique that maximizes variance without considering class labels.

The quantitative metrics support these observations: Between-class distance was 2.7494 indicating a moderate separation between class centroids. Within-class scatter was 0.9664, a reasonable compactness within classes. Separation ratio was 2.8451 indicating moderate overall class separation

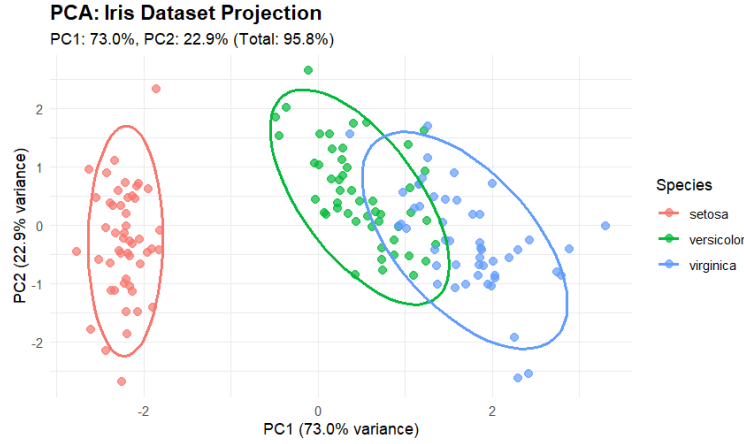


Figure 2: Iris PCA Plot

In contrast, LDA achieves superior class discrimination. The first linear discriminant (LD1) accounts for 99.12% of the discrimination, with LD2 contributing only 0.88%. This indicates that nearly all class separation occurs along a single axis, which aligns with the mathematical expectation that LDA can produce at most $c-1$ discriminant directions (where $c = 3$ classes, thus 2 directions maximum). The LDA visualization shows minimal to no overlap between the three species classes. Each class forms a distinct, well-separated cluster based on centroid values; Setosa: $LD1 = 7.61$ (far right). Versicolor: $LD1 = -1.83$ (center). Virginica: $LD1 = -5.78$ (far left)

The quantitative superiority of LDA is evident:

Between-class distance: 9.0068 - **3.3** $\times$ greater than PCA
 Within-class scatter: 2.0000 - comparable to PCA
 Separation ratio: 4.5034 - **1.58** $\times$ better than PCA

This demonstrates that LDA, as a supervised technique, explicitly optimizes the objective function to maximize between-class variance while minimizing within-class variance, directly implementing the mathematical framework:

$$\max \frac{v^T S_b v}{v^T S_w v}$$

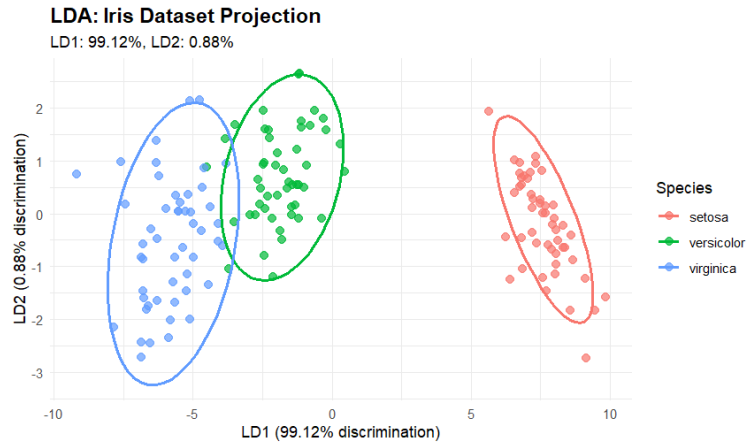


Figure 3: Iris LDA Plot

4.2 MNIST Dataset

The simulated MNIST analysis (784 dimensions, 1000 samples) demonstrates PCA's effectiveness in high-dimensional spaces. To capture the same 95% of the variance as in the Iris Dataset, only 187 components are needed out of 784, representing a **76.15%** dimensionality reduction.

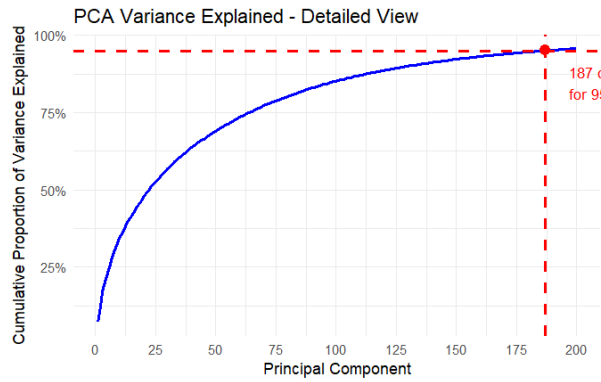


Figure 4: PCA Cumulative Variance Captured Plot

When analyzing visually the plots of LDA and PCA onto the first two components it is clear that LDA is a far superior class distinction method. LDA boasts a 1.82 class separation ratio, compared to PCA's which is 0.603. Within the first two components LDA betters PCA by about **201.82%**

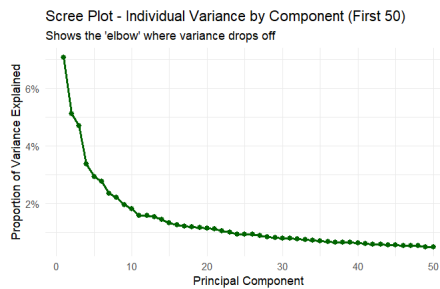


Figure 5: Individual Variance Captured PCA

On the other hand, the analysis exposed LDA's critical limitation in high-dimensional spaces, the singularity of the within-class scatter matrix S_w , the matrix is not invertible, meaning its determinant is

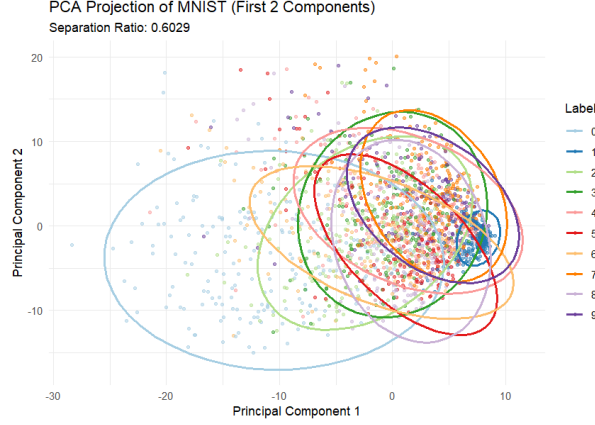


Figure 6: PCA Projection, First Two Components, MNIST

zero and the eigenvalue problem becomes unsolvable. Consider a dataset with n samples and d dimensions. The rank of S_w is at most $\min(n, d)$. If $n < d$, this makes the S_w matrix singular. Rank determines the number of linearly independent rows or columns, dictating the amount of nonzero eigenvalues, and the centered data often does not span all d dimensions.

If invertibility of a matrix is not possible, then the:

$$S_w^{-1} S_b v = \lambda v$$

equation is not solvable since S_w cannot be inverted.

Within the MNIST dataset Number of samples (n) = 1000 Number of features (d) = 784 The rank of $S_w \leq \min(n - c, d) = \min(990, 784) = 784$

Since we have $n > d$ in this case ($1000 > 784$), technically S_w should be invertible. However, within the actual MNIST dataset structure, the calculated value for, take for example the 627th eigenvalue is 2.325×10^{-27} , which is essentially zero for computational purposes.

Additionally, LDA struggled at times with nonlinear separation since it is limited to $c - 1$ discriminatory directions, a limitation that PCA does not have due to no dimensional constraints, and its nature as a non supervised model.

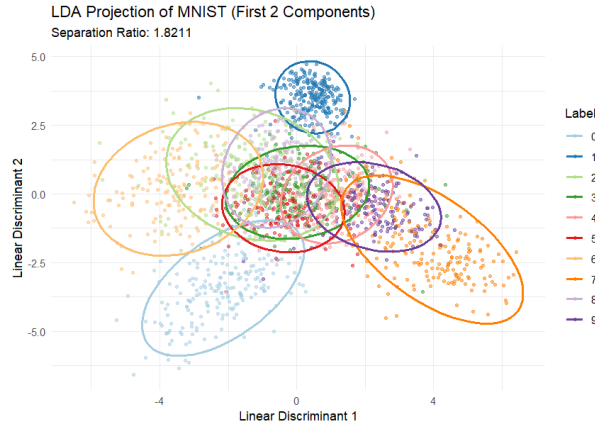


Figure 7: LDA Projection, First Two Components, MNIST

5 Conclusion

Dimensionality reduction bridges linear algebra, statistics, and machine learning. Through this investigation it has become clear that the choice between PCA and LDA is not about which method is better but rather which objective the researcher wishes to achieve

When the primary objective is data compression with maximum variance retention, common in data analysis, image preprocessing or noise reduction, PCA is highly effective. The PCA function was able to reduce 784 to only 187 principal components (24% of original) while retaining 95% of the variance, demonstrating PCA's strength in high-dimensional settings. In contrast, when the task involves classification or the data contain known class labels, LDA consistently outperforms PCA in separability. On the Iris dataset, LDA achieved a separation ratio 58 % higher than PCA and over triple on the MNIST dataset. Producing almost perfect visual class separation in two dimensions on the Iris set, whereas PCA exhibited noticeable overlap between Versicolor and Virginica . This superiority stems directly from LDA's optimization of $\frac{v^T S_b v}{v^T S_w v}$, which explicitly maximizes between-class scatter while minimizing within-class scatter.

However, the empirical analysis also exposed LDA's practical limitations in very high-dimensions, the within-class scatter matrix S_w becomes rank-deficient or numerically singular when the number of features greatly exceeds the effective sample size per class. This singularity forces researchers to adopt hybrid approaches. Usually applying PCA first to reduce dimensionality below the sample size, then applying LDA, a method now common in facial recognition analysis. Ultimately, neither technique is universally superior. PCA excels at discovering the intrinsic geometric structure of data independent of labels, while LDA exploits supervisory information to find the most discriminative subspace. The most effective strategy therefore depends on the research question itself: if the aim is a representation of variance, choose PCA; if the aim is linear separation of known classes, choose LDA (or PCA+LDA when $p > n$).

Through this investigation, it is clear that in modern data science a deep mathematical understanding is foundational to the application of each tool. Reduction techniques such as PCA and LDA offer a wide range of applications across various industries, including gene classification, financial market modeling and many more. It is inarguable that future developments in various industries will be contingent on the progress of underlying mathematical models, helping us understand the multi-dimensional world we live in.

Works Cited

- Builtin. “A Step-by-Step Explanation of Principal Component Analysis (PCA).” Builtin, <https://builtin.com/data-science/step-step-explanation-principal-component-analysis>.
- Chen, Guangliang. “Lecture 11: Linear Discriminant Analysis.” San José State University, <https://www.sjsu.edu/faculty/guangliang.chen/Math253S20/lec11lda.pdf>. Accessed 12 Dec. 2025
- Fisher, R. A. “Iris.” UCI Machine Learning Repository, 1 July 1988, <https://archive.ics.uci.edu/dataset/53/iris>.
- “Introduction to Principal Component Analysis (PCA).” YouTube, uploaded by StatQuest with Josh Starmer, 12 Mar. 2018, <https://www.youtube.com/watch?v=g-Hb26agBFg>.
- Kuttler, Kenneth. “Eigenvalues and Eigenvectors of a Matrix.” A First Course in Linear Algebra, LibreTexts, 2023, [https://math.libretexts.org/Bookshelves/Linear_Algebra/A_First_Course_in_Linear_Algebra_\(Kuttler\)/07](https://math.libretexts.org/Bookshelves/Linear_Algebra/A_First_Course_in_Linear_Algebra_(Kuttler)/07)
- “Laplace Expansion.” Wikipedia, Wikimedia Foundation, https://en.wikipedia.org/wiki/Laplace_expansion. “Linear Discriminant Analysis in R .” *R – bloggers*, 11 May 2021, <https://www.r-bloggers.com/2021/05/linear-discriminant-analysis-in-r/>.
- “Linear Transformations.” Interactive Linear Algebra, Georgia Institute of Technology, <https://textbooks.math.gatech.edu/ila/linear-transformations.html>.
- “PCA - Principal Component Analysis Essentials.” DataCamp, <https://www.datacamp.com/tutorial/pca-analysis-r>.
- StatQuest with Josh Starmer. “StatQuest: PCA - Practical Tips.” YouTube, 16 Apr. 2018, <https://www.youtube.com/watch?v=ekZQthaCrFU>.
- 3Blue1Brown. “Eigenvectors and Eigenvalues — Chapter 14, Essence of Linear Algebra.” YouTube, 26 Aug. 2016, <https://www.youtube.com/watch?v=PFDu9oVAE-g>.
- 3Blue1Brown. “Linear Transformations and Matrices — Chapter 3, Essence of Linear Algebra.” YouTube, 5 Aug. 2016, <https://www.youtube.com/watch?v=uQhTuRlWMxw>.
- Wortman, James. “Block Matrix Multiplication.” University of Utah Math 1050, <https://www.math.utah.edu/~wortman/1050-text-2b2m.pdf>.
- “Understanding Covariance and Correlation Matrices.” YouTube, uploaded by ritvikmath, 26 Oct. 2020, <https://www.youtube.com/watch?v=ekZQthaCrFU>.
- “Visually Explained: Laplace Expansion for the Determinant.” YouTube, uploaded by Dr. Trefor Bazett, 8 Mar. 2021, <https://www.youtube.com/watch?v=3EaVlzOGPos>.